01211373 Machine Learning and Programming for Industry

Lecture 12 Supplement 1

Teaching a quadrotor to hover

From Bellman's equation to DQN, PPO, SAC and sim-to-real

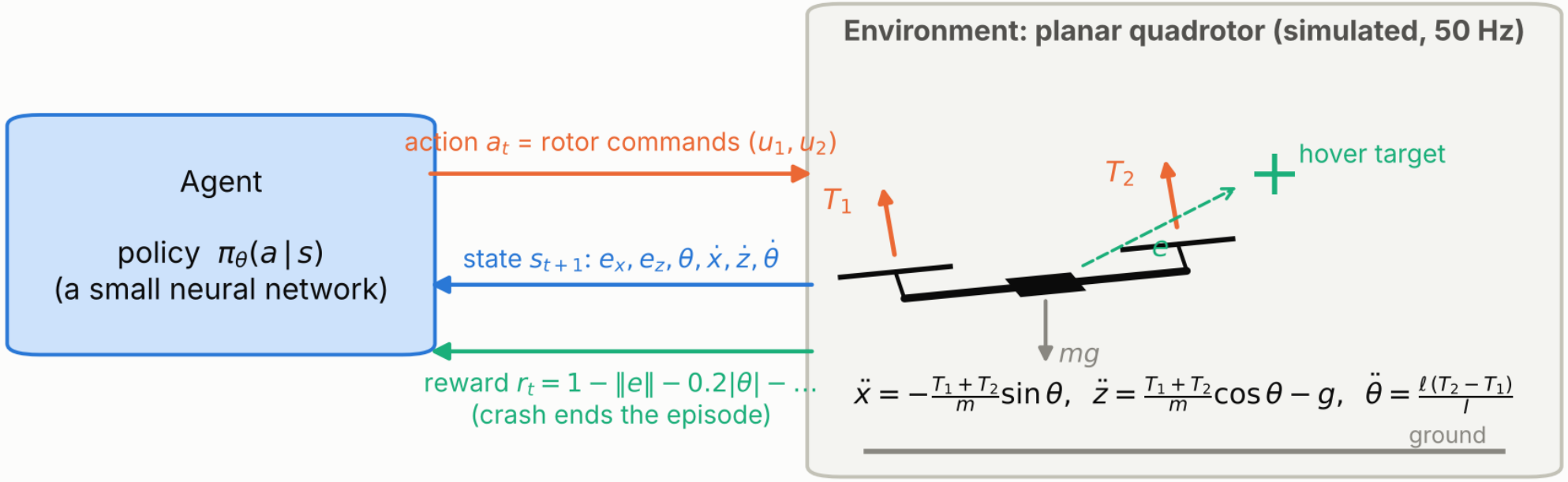
- The control task as a Markov decision process
- Value iteration and tabular Q-learning on altitude hold
- Deep Q-networks with discrete thrust levels
- Policy gradients and PPO with continuous rotor commands
- Off-policy actor-critic (SAC) and sample efficiency
- Domain randomisation for sim-to-real
- Where the field is going

Companion notebook: `mlpi26_lect12_nbsup1.ipynb`

THE SETTING

The quadrotor as a decision problem

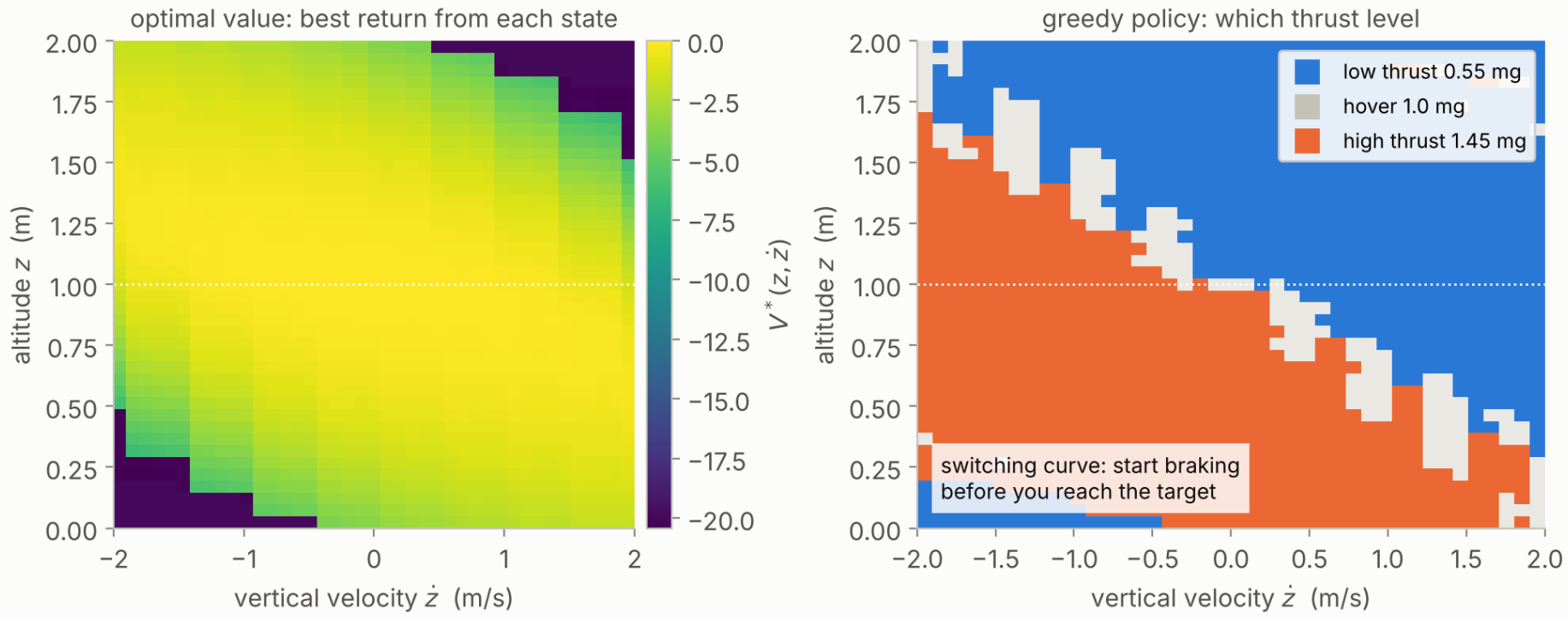
- An agent picks rotor commands, the simulator applies Newton's laws for 20 ms, and a scalar reward says how close, upright and calm the vehicle is.
- Nobody supplies the correct thrust: the agent must discover which actions lead to high reward over the next few seconds.



You choose the state, the action space, the reward and what ends an episode. That choice is the controller design; the algorithm only optimises it.

Bellman's equation, solved by value iteration

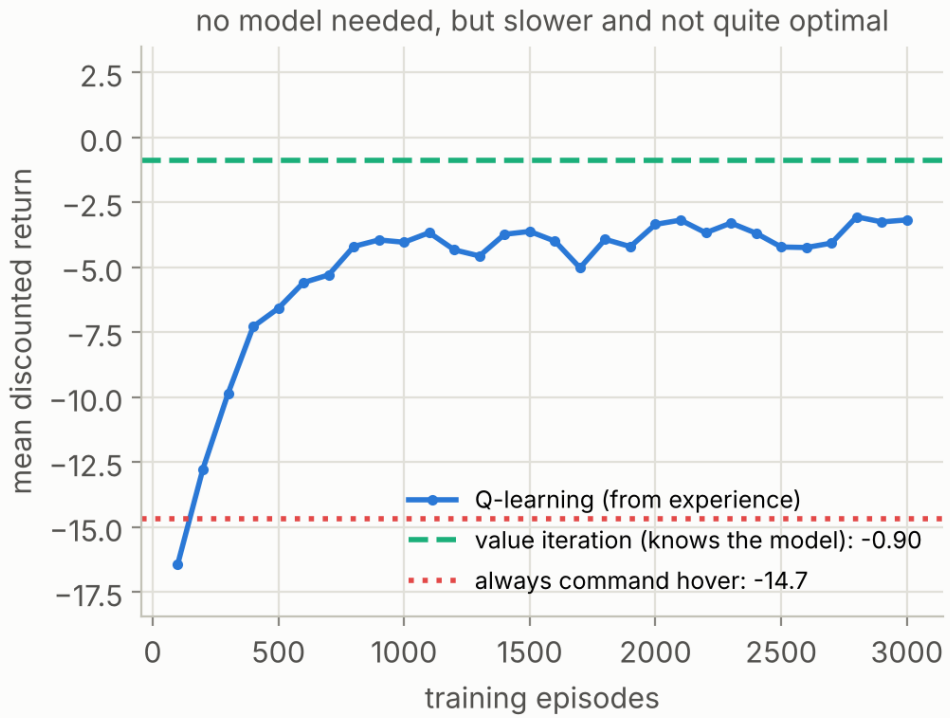
- V*(s) = max over a of [r(s,a) + γ · V*(s′)]: the value of a state is the best immediate reward plus the discounted value of where it leaves you.
- On the 1-D altitude task (41 × 41 grid, 3 thrust levels) iterating this update converges in 209 sweeps; the greedy policy is then optimal.

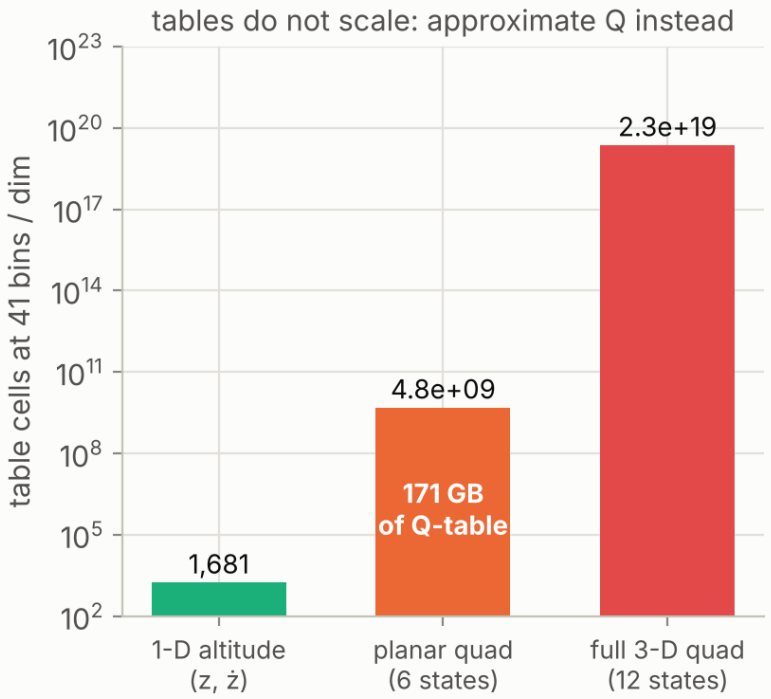


Value iteration rediscovers a bang–bang switching curve: brake before the target. Optimal return -0.90, versus -14.7 for "always hover".

Q-learning learns from flights, not from a model

- Replace the expectation with one observed transition: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$, exploring with ε-greedy.
- It is off-policy: the target uses the greedy action, so the agent can explore randomly and still learn the greedy policy.

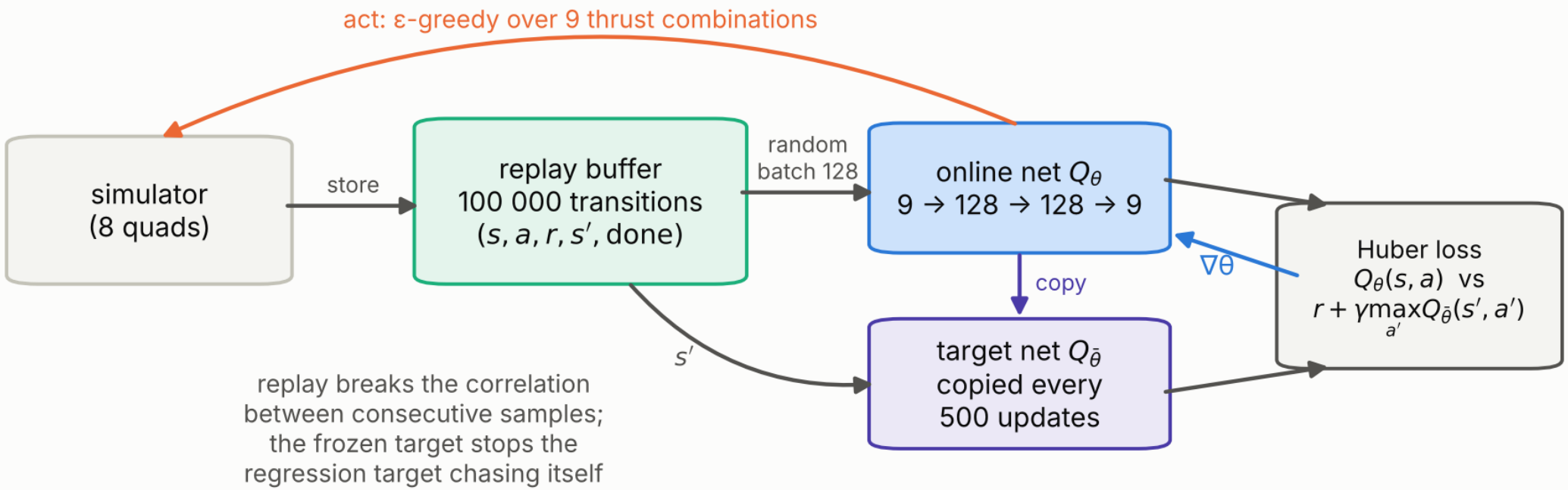




Tabular Q-learning reaches -3.17 (model-based optimum -0.90). The same table for the 6-state planar quad would need 171 GB.

DQN: a neural network instead of a table

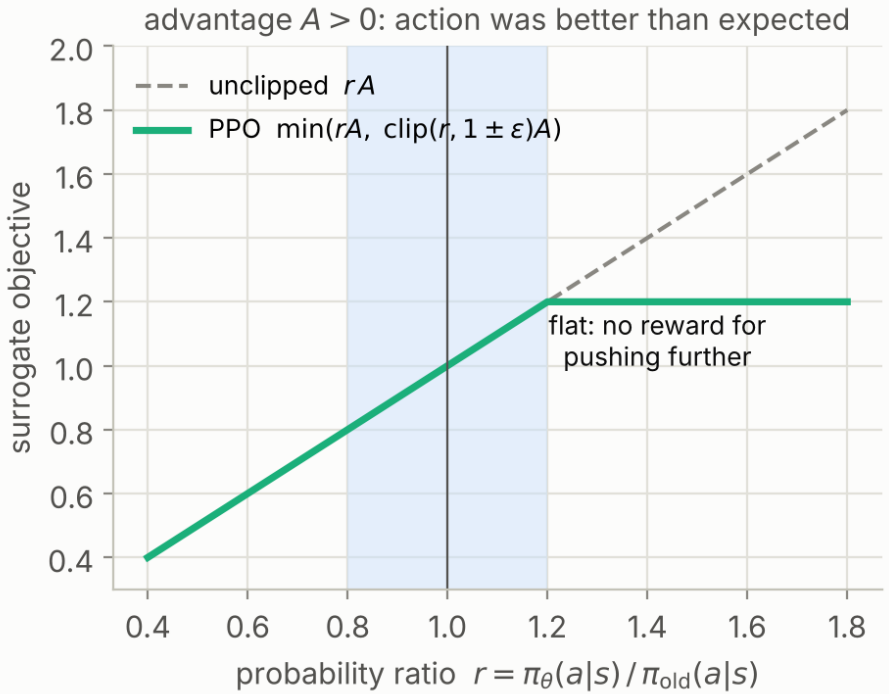
- Qθ(s, ·) outputs a value for each of 9 discrete rotor-command pairs (3 collective × 3 differential); training minimises the TD error.
- Naive training diverges. A replay buffer decorrelates samples, and a frozen target network stops the regression target from chasing itself.

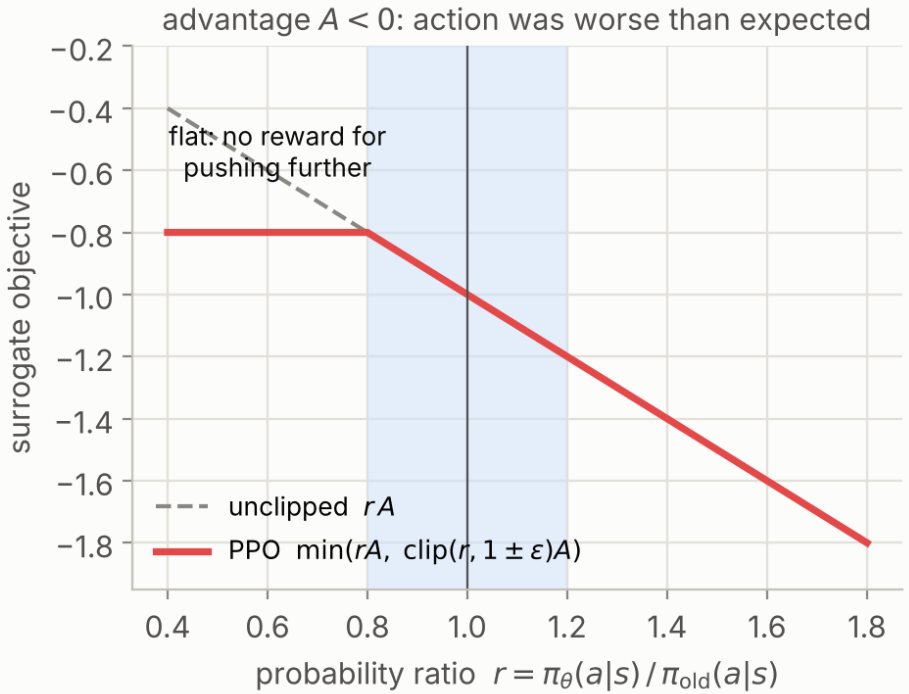


After 240k steps (66 s CPU) DQN hovers: return 204.0, 98 % success. Always bootstrap through a time-out; only a crash is terminal.

Policy gradients, and why PPO clips

- $\nabla J = E[\, \nabla \log \pi\theta(a|s) \cdot \hat{A}\,]$: raise the probability of actions that did better than the critic expected. GAE estimates the advantage $\hat{A}$ from the critic's TD errors.
- A large step can wreck the policy, and a crashing policy collects no useful data. PPO reuses each batch for several epochs but removes the reward for moving far.

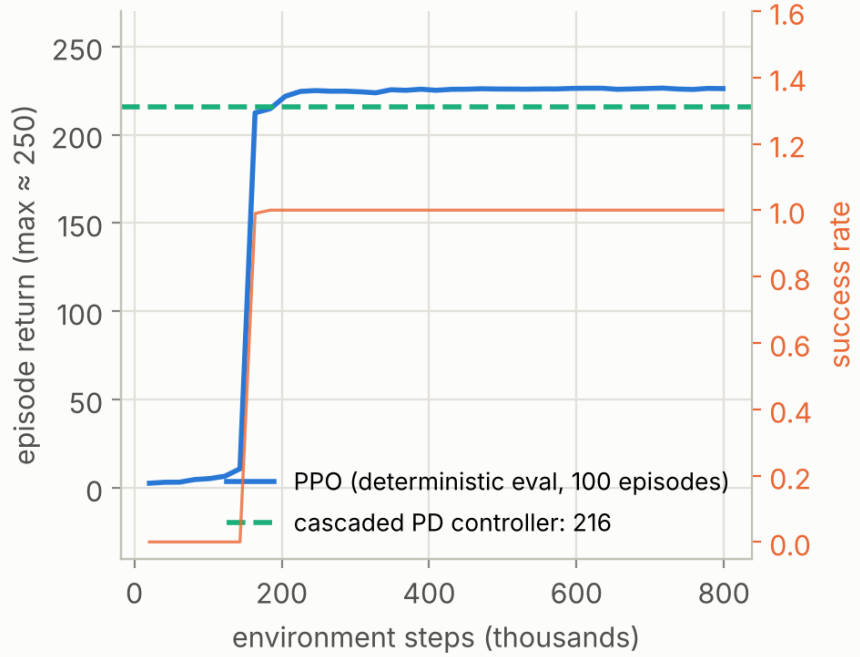


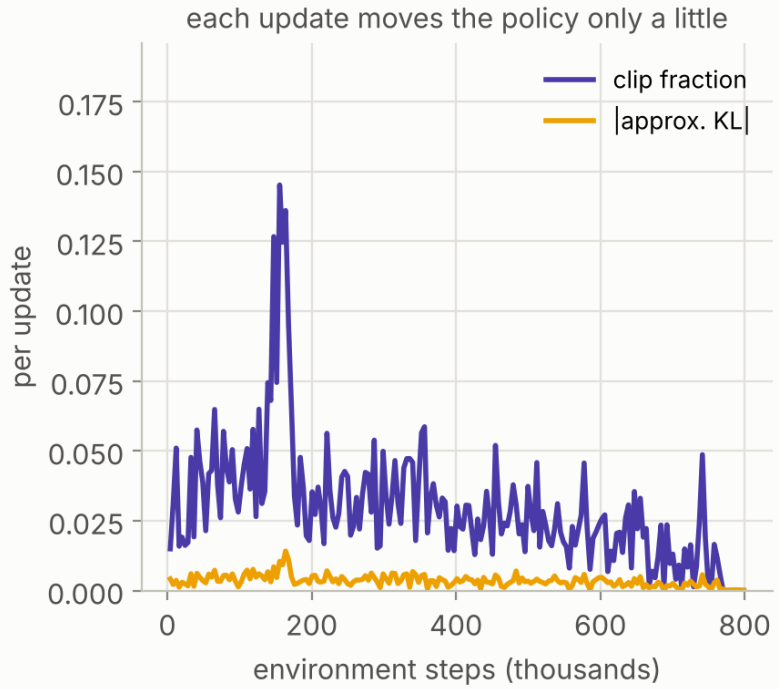


The min() only removes optimism: gains beyond the 1 ± ε band are ignored, losses are not. That is a cheap trust region.

PPO learns to hover in 164k steps

- 16 parallel quads × 256 steps per batch, 10 epochs, ε = 0.2, GAE λ = 0.95, actor and critic each 2 × 64 tanh, continuous rotor commands.





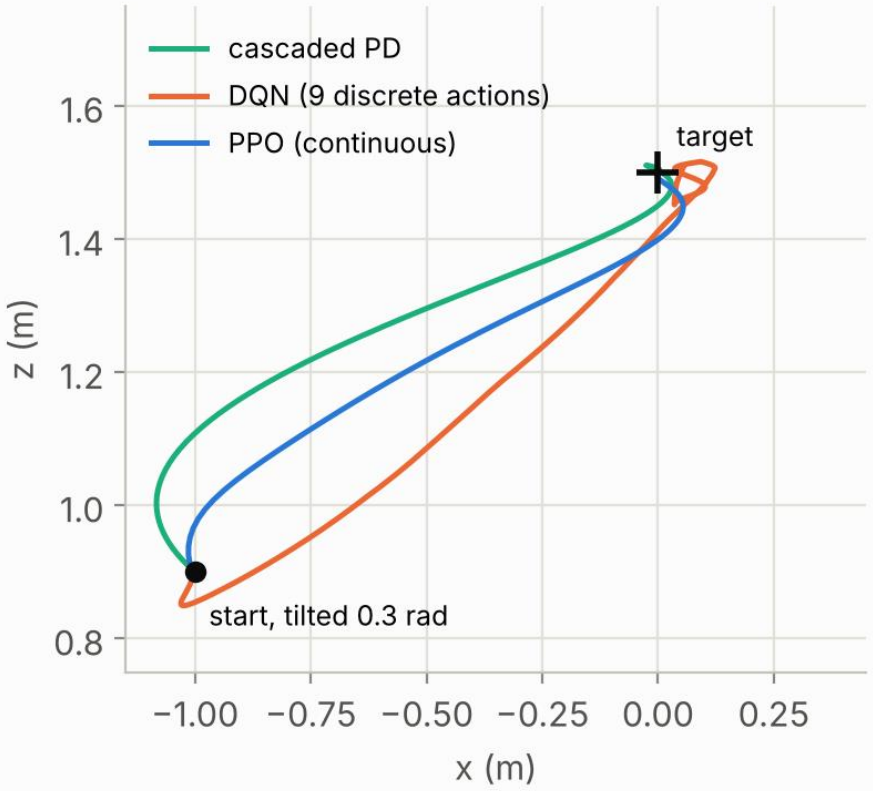

```
ratio = (logp_new - logp_old).exp()
loss_pi = -torch.min(ratio*adv, ratio.clamp(0.8, 1.2)*adv).mean()
loss = loss_pi + 0.5 * F.mse_loss(V(obs), returns)
```

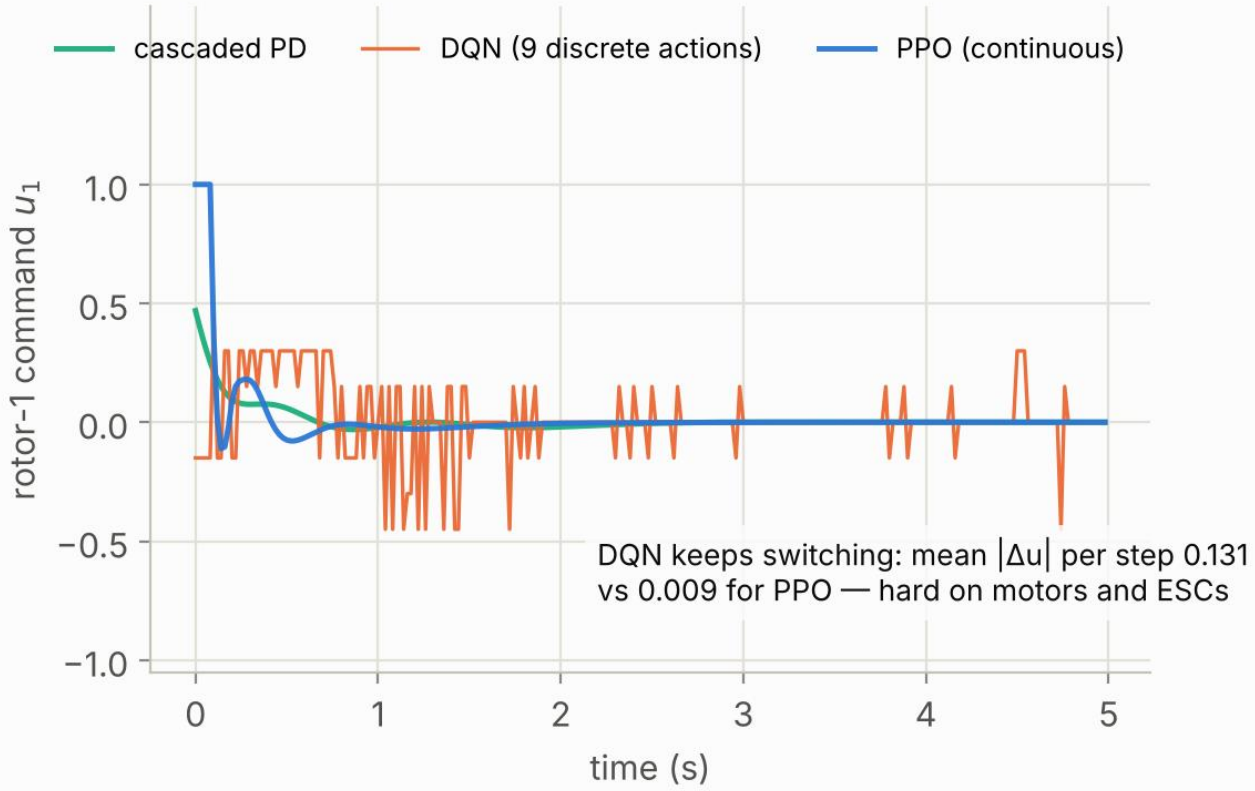
Return 225.8 vs PD 215.8, 100 % success, 64 s CPU.
Clip fraction ≈ 0.03: healthy.

COMPARING THE POLICIES

Three controllers, one difficult start

- Start 1 m left, 0.6 m low and tilted 0.3 rad. All three reach the target; the difference is in how they command the motors.



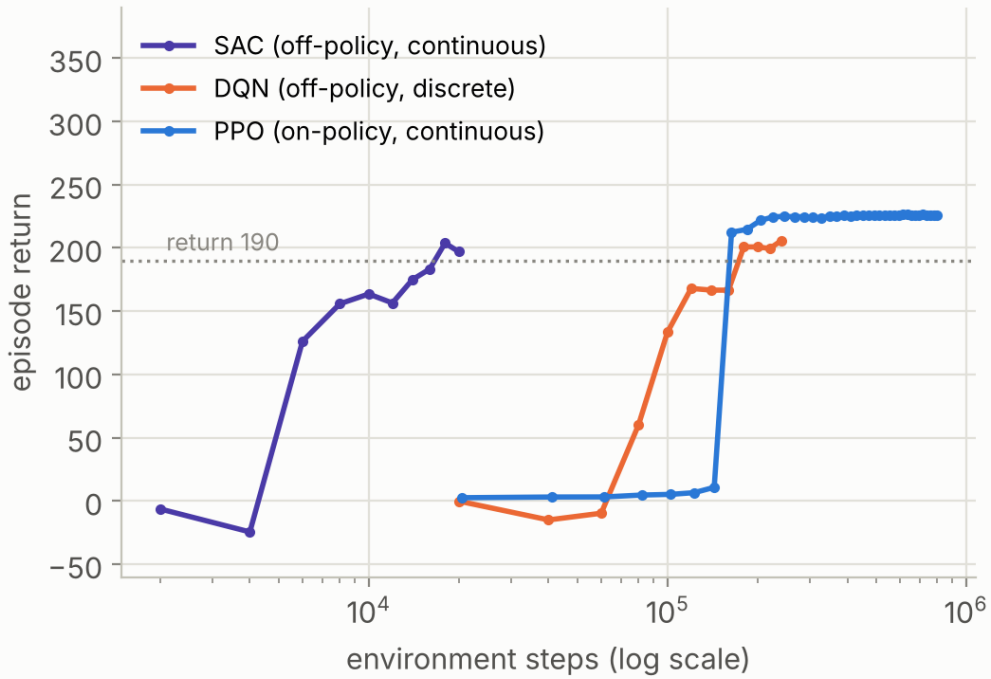


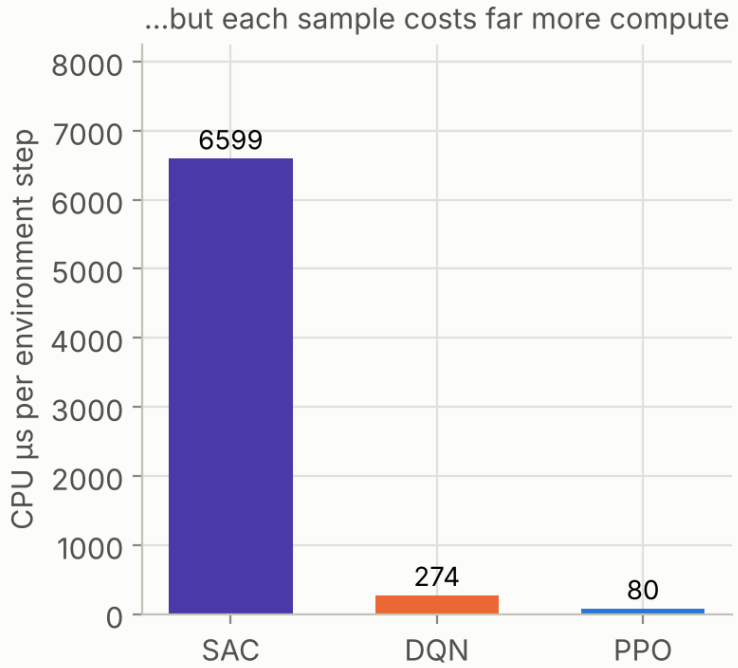
DQN hovers by switching between thrust levels (15× PPO's command activity, 7.0 cm limit cycle). Discretise only when the actuator really is discrete.

SAMPLE EFFICIENCY

Off-policy SAC: fewer samples, more compute

- SAC keeps a replay buffer like DQN but has a continuous actor, twin Q-critics and an entropy bonus whose weight α is tuned automatically.
- PPO discards each batch after 10 epochs; SAC updates its three networks after every environment step.



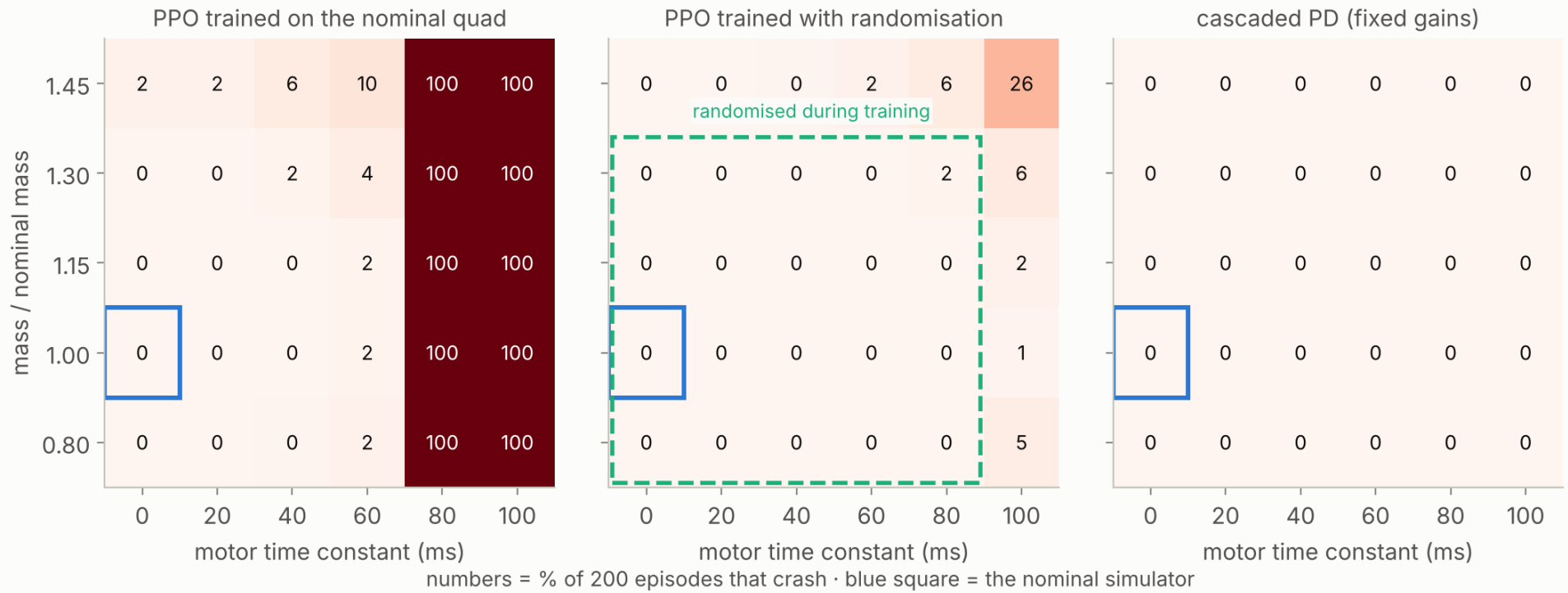


Return 190 after 18k steps for SAC vs 164k for PPO, but 6599 vs 80 µs of CPU per step. Real-robot time favours SAC; a fast simulator favours PPO.

SIM-TO-REAL

A policy that only works in its own simulator

- Test on quads that differ from the simulator: mass 0.8–1.45 × nominal and motor time constants 0–100 ms, 200 episodes per cell.
- Domain randomisation: every training episode draws mass, inertia, motor strength and a 0–80 ms motor lag, so one policy must fly them all.

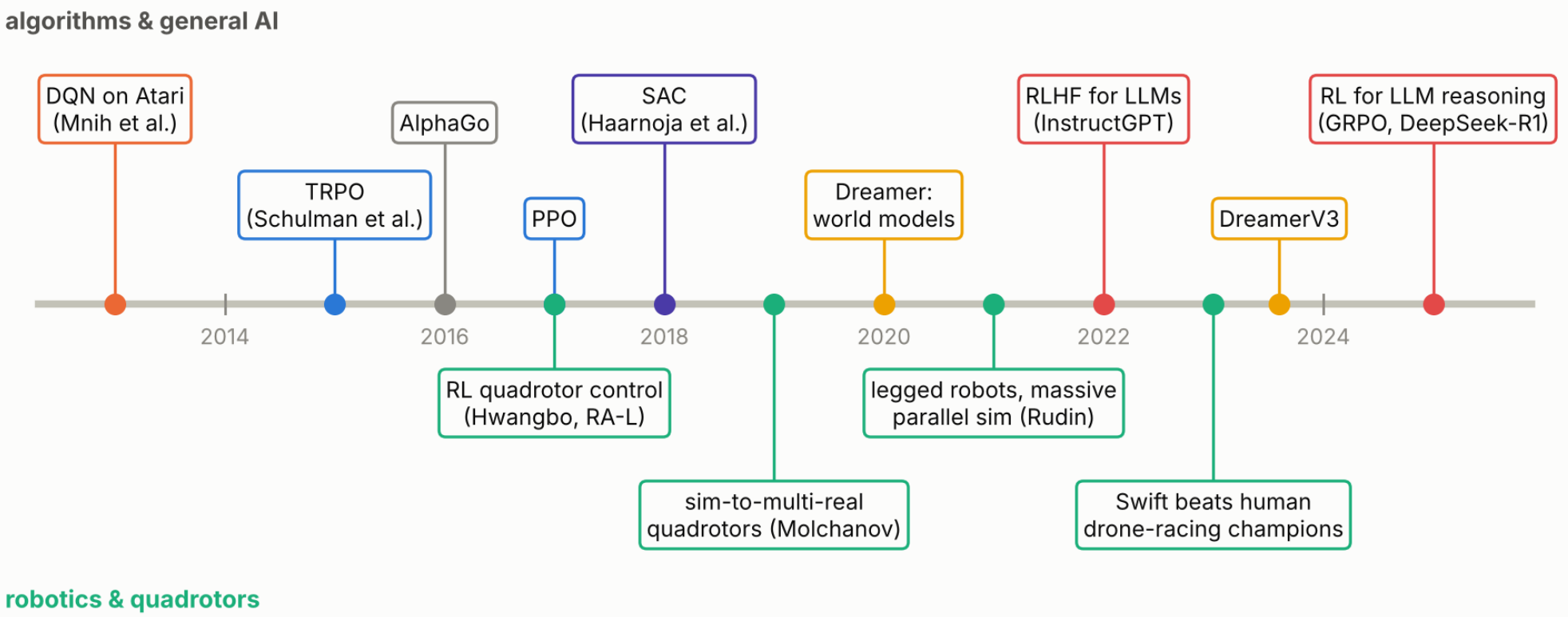


At 80 ms motor lag the nominal policy crashes 100 % of flights, the randomised one 0 %, for a nominal return of 224.1 vs 225.8.

RECENT ADVANCES

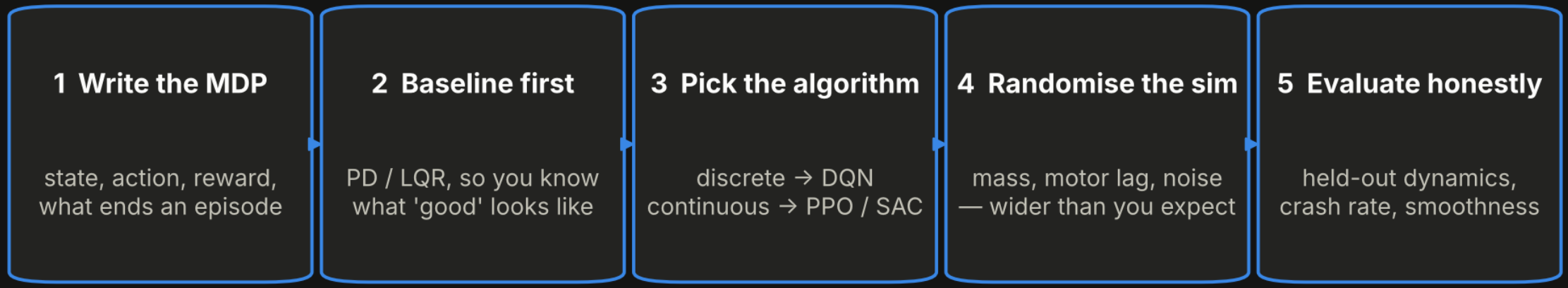
Where the field has gone since DQN

- Methods from games and language now train real robots: PPO for simulation-trained flight and locomotion, SAC for learning on hardware.



The pattern behind the robotics results: massive parallel simulation, randomise what you cannot model, then correct with real-world data.

What to take away



1. Bellman's equation is the core of every method here; DQN, PPO and SAC differ in what they approximate and how they collect data.
2. Tables do not scale past a few states: approximate Q or the policy with a network, and stabilise it (replay, target nets, clipping).
3. Match the action space to the actuator — discrete actions on a motor chatter.
4. A time-out is not a crash: bootstrap through truncation.
5. Fast parallel simulator → PPO; scarce real-robot data → SAC.
6. Randomise mass, latency and motor strength wider than your uncertainty, and test outside that range.
7. Report crash rate, error and smoothness against a classical baseline, not only the return.

Generated by Claude and customized by

dew.ninja 2026

Notebook: mlpi26_lect12_nbsup1.ipynb — code, all experiments, 8 exercises and references (Sutton & Barto 2018; Mnih 2015; Schulman 2017; Haarnoja 2018; Kaufmann 2023).